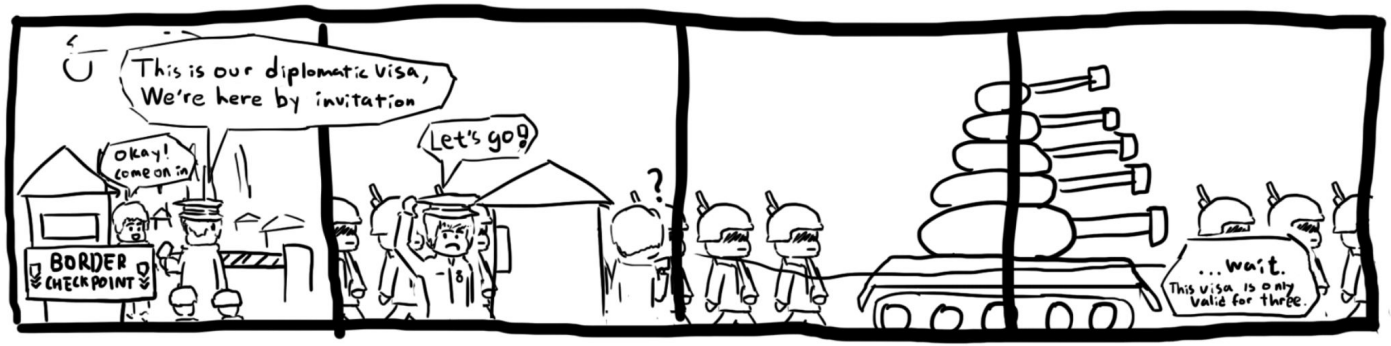
Memory Safety Vulnerabilities

CS 161 Spring 2026 - Lecture 3



Last Time: x86 Assembly and Call Stack

- C memory layout
  - **Code** section: Machine code (raw bits) to be executed
  - **Static** section: Static variables
  - **Heap** section: Dynamically allocated memory (e.g. from `malloc`)
  - **Stack** section: Local variables and stack frames
- x86 registers
  - **EBP** register points to the top of the current stack frame
  - **ESP** register points to the bottom of the stack
  - **EIP** register points to the next instruction to be executed
- x86 calling convention
  - When calling a function, the old EIP (RIP) is saved on the stack
  - When calling a function, the old EBP (SFP) is saved on the stack
  - When the function returns, the old EBP and EIP are restored from the stack

Today: Memory Safety Vulnerabilities

- Buffer overflows
  - Stack smashing
  - Memory-safe code
- Integer memory safety vulnerabilities
- Format string vulnerabilities
- Heap vulnerabilities
- Writing robust exploits

Buffer Overflow Vulnerabilities



Textbook Chapter 3.1

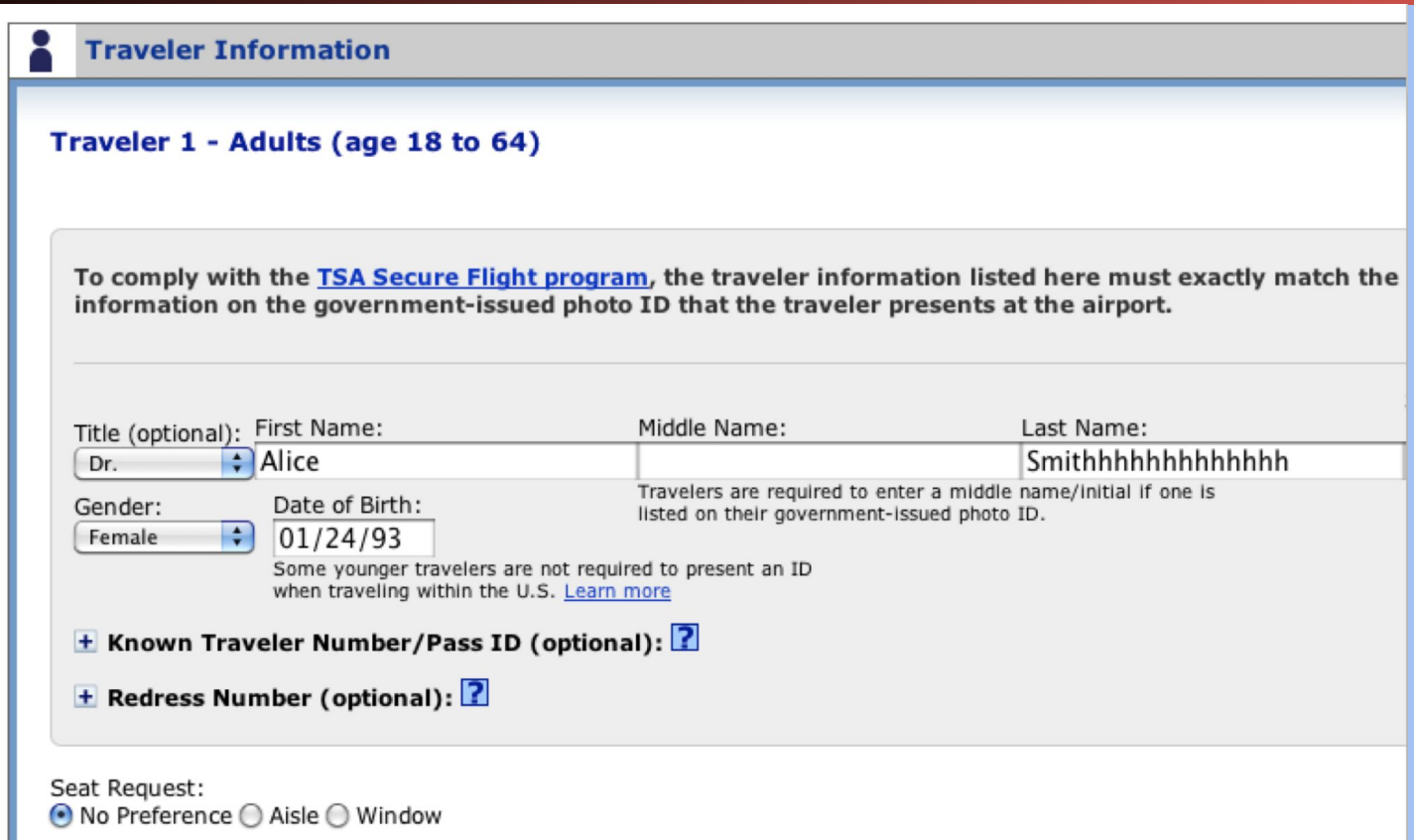
Consider an Airport Terminal…



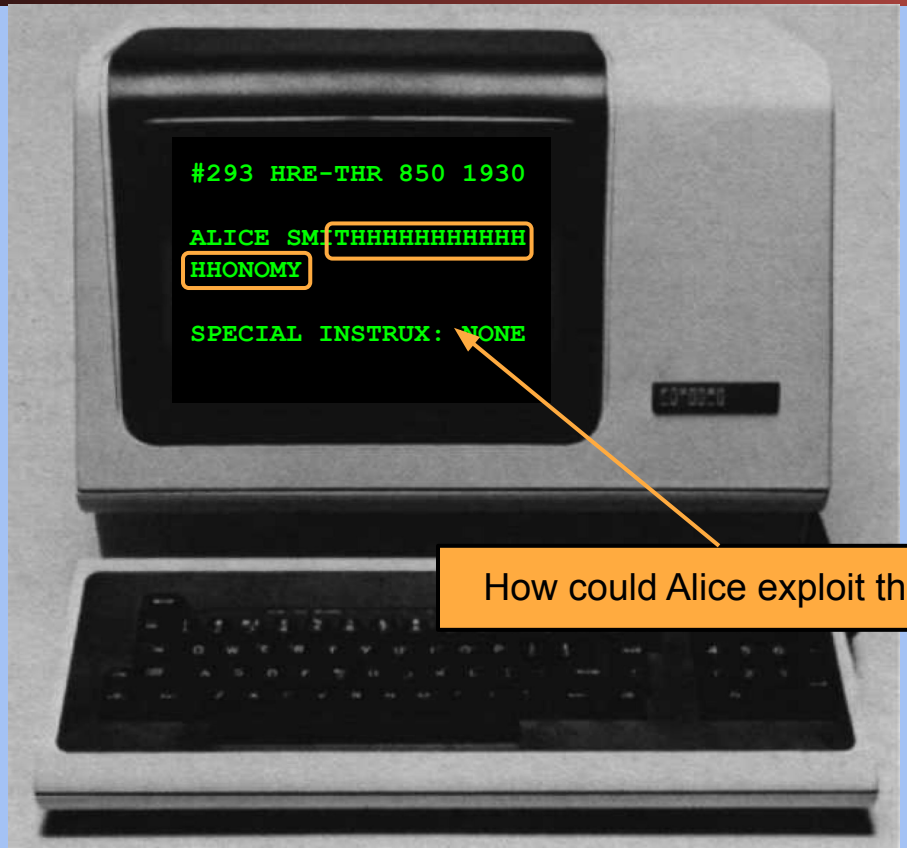
Consider an Airport “Terminal”…



Consider an Airport “Terminal”…

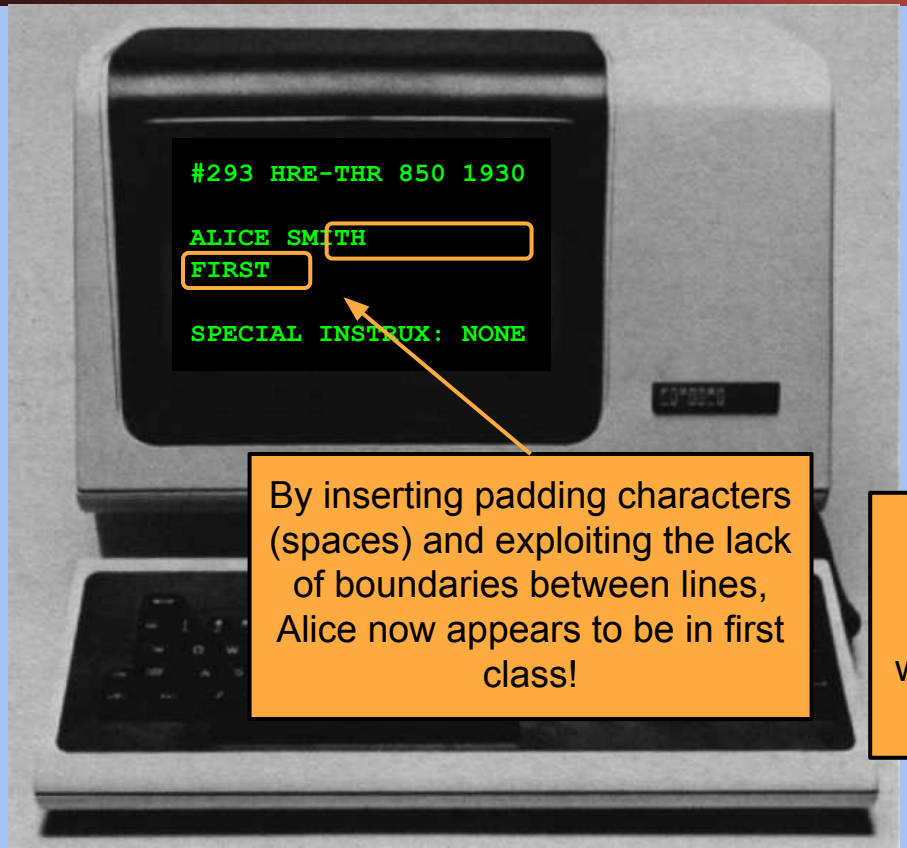


Consider an Airport “Terminal”…



Consider an Airport “Terminal”…





By inserting padding characters (spaces) and exploiting the lack of boundaries between lines, Alice now appears to be in first class!

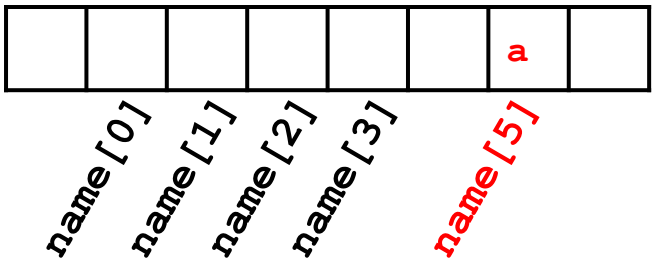
Takeaway: Attackers can exploit lack of boundaries to control areas (memory, as we will see shortly) that they aren’t supposed to control

Buffer Overflow Vulnerabilities

- Recall: C has no concept of array length; it just sees a sequence of bytes
- If you allow an attacker to start writing at a location and don't define when they must stop, they can overwrite other parts of memory!

```
char name[4];
name[5] = 'a';
```

This is technically valid C code, because C doesn't check bounds!



Vulnerable Code


```
char name[20];

void vulnerable(void) {
    ...
    gets(name);
    ...
}
```

The **`gets`** function will write bytes until the input contains a newline (**`'\n'`**), *not* when the end of the array is reached!

Okay, but there's nothing to overwrite—for now…

Vulnerable Code


```
char name[20];
char instrux[20] = "none";

void vulnerable(void) {
    ...
    gets(name);
    ...
}
```

What does the memory diagram of static data look like now?

Vulnerable Code

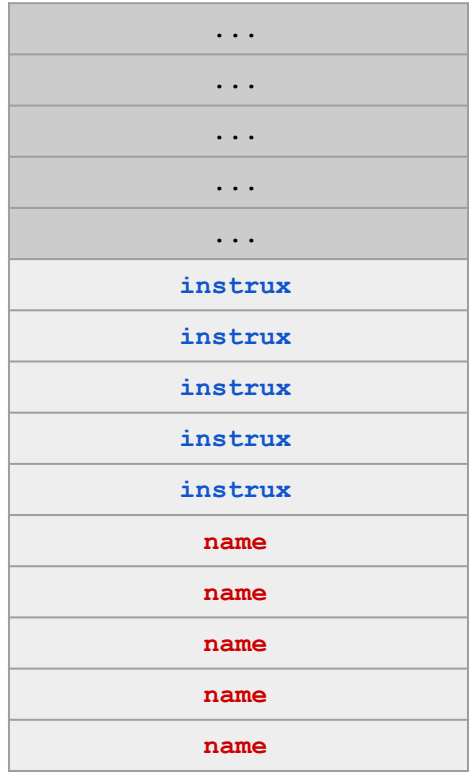
What can go wrong here?

gets starts writing here and can overwrite anything above **name**!

```
char name[20];
char instrux[20] = "none";

void vulnerable(void) {
    ...
    gets(name);
    ...
}
```

Note: **name** and **instrux** are declared in static memory (outside of the stack), which is why **name** is below **instrux**



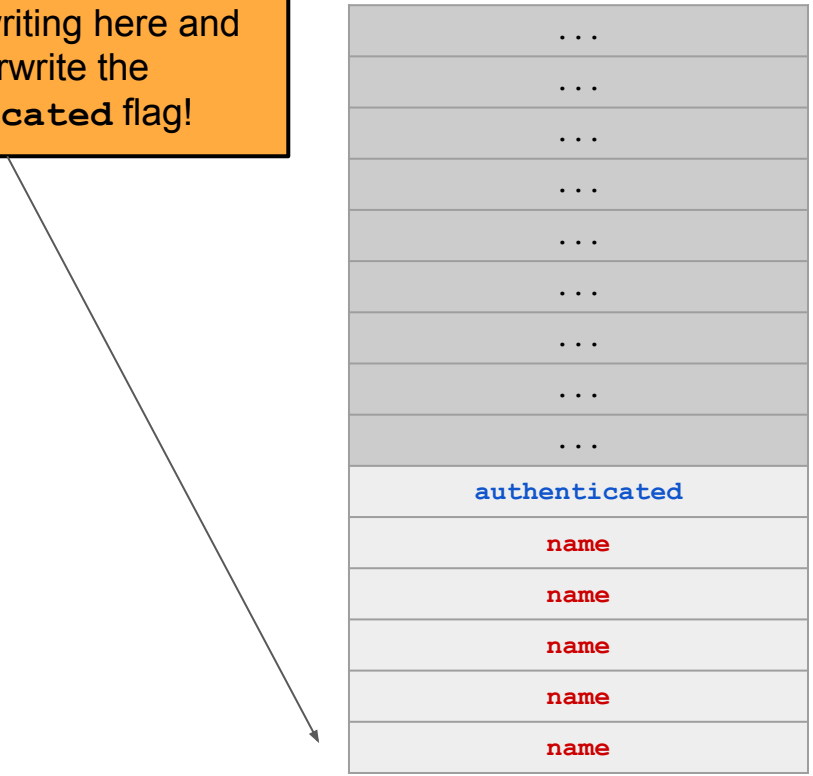
Vulnerable Code

What can go wrong here?

`gets` starts writing here and can overwrite the `authenticated` flag!

```
char name[20];
int authenticated = 0;

void vulnerable(void) {
    ...
    gets(name);
    ...
}
```

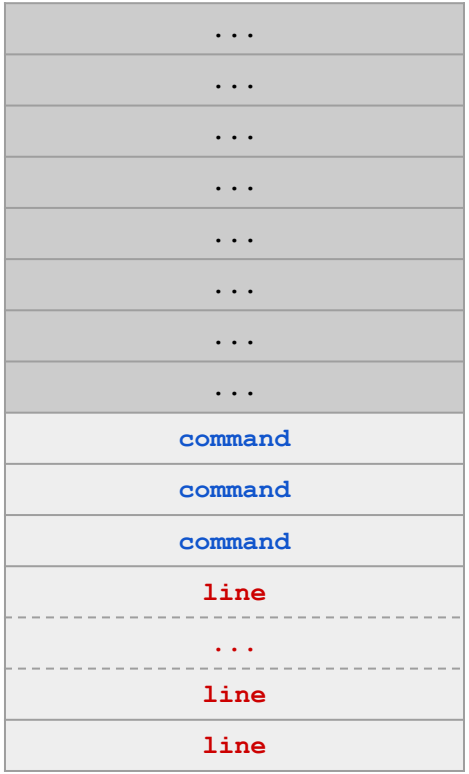


Vulnerable Code

What can go wrong here?

```
char line[512];
char command[] = "/usr/bin/ls";

int main(void) {
    ...
    gets(line);
    ...
    execv(command, ...);
}
```



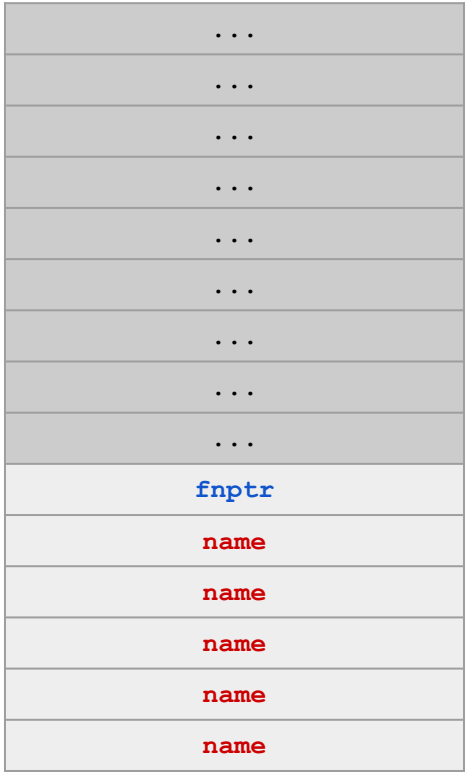
Vulnerable Code

What can go wrong here?

fnptr is called as a function, so the EIP jumps to an address of our choosing!

```
char name[20];
int (*fnptr)(void);

void vulnerable(void) {
    ...
    gets(name);
    ...
    fnptr();
}
```



Top 10 Most Dangerous Software Weaknesses (2023)

Rank	ID	Name	Score
[1]	CWE-787	Out-of-bounds Write	63.72
[2]	CWE-79	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	45.54
[3]	CWE-89	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')	34.27
[4]	CWE-416	Use After Free	16.71
[5]	CWE-78	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	15.65
[6]	CWE-20	Improper Input Validation	15.50
[7]	CWE-125	Out-of-bounds Read	14.60
[8]	CWE-22	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	14.11
[9]	CWE-352	Cross-Site Request Forgery (CSRF)	11.73
[10]	CWE-434	Unrestricted Upload of File with Dangerous Type	10.41



Note: Python Syntax

- For this class, you will see Python syntax used to represent sequences of bytes
  - This syntax will be used in Project 1 and on exams!
- Adding strings: Concatenation
  - `'abc' + 'def' == 'abcdef'`
- Multiplying strings: Repeated concatenation
  - `'a' * 5 == 'aaaaa'`
  - `'cs161' * 3 == 'cs161cs161cs161'`

Note: Python Syntax

- Raw bytes
  - `len('\xff') == 1`
- Characters can be represented as bytes too
  - `'\x41' == 'A'`
- Note for the project: `'\\'` is a literal backslash character
  - `len('\\xff') == 4`, because the slash is escaped first
    - This is a literal slash character, a literal `'x'` character, and 2 literal `'f'` characters
    - `'\\xff' == '\x5c\x78\x66\x66'`

Stack Smashing



Textbook Chapter 3.2

Stack Smashing

- The most common kind of buffer overflow
- Occurs on stack memory
- What are some values on the stack an attacker can overflow?
  - Local variables
  - Function arguments
  - Saved frame pointer (SFP)
  - Return instruction pointer (RIP)

- Recall: When returning from a program, the EIP is set to the value of the RIP saved on the stack in memory
  - Like the function pointer, this lets the attacker choose an address to jump (return) to!

Overwriting the RIP

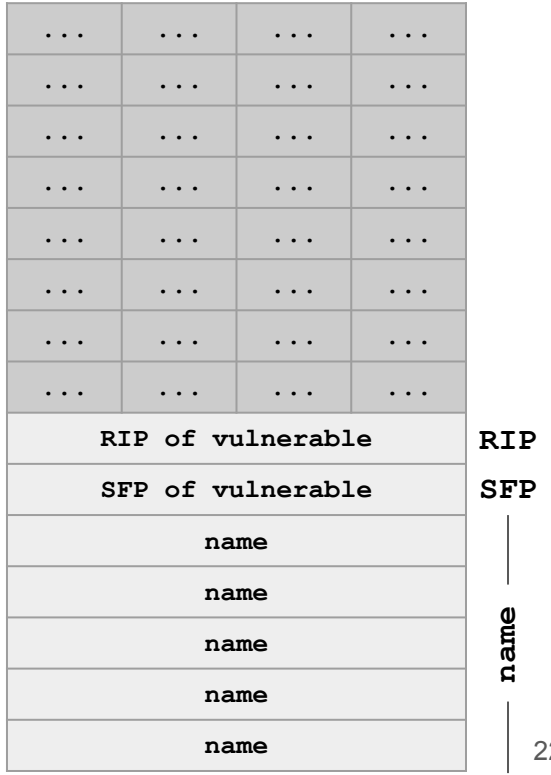
Assume that the attacker wants to execute instructions at address `0xdeadbeef`.

What value should the attacker write in memory? Where should the value be written?

What should an attacker supply as input to the `gets` function?

```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```

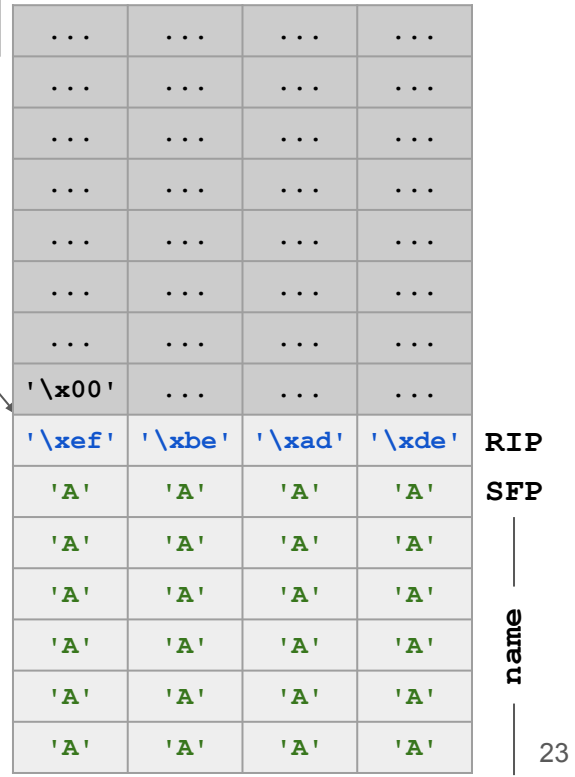
`gets` starts writing here and can overwrite anything above `name`, including the RIP!



Overwriting the RIP

- Input: `'A' * 24 + '\xef\xbe\xad\xde' + '\n'`
  - 24 garbage bytes to overwrite all of **name** and the SFP of **vulnerable**
  - The address of the instructions we want to execute
    - Remember: Addresses are little-endian!
- What if we want to execute instructions that aren't in memory?

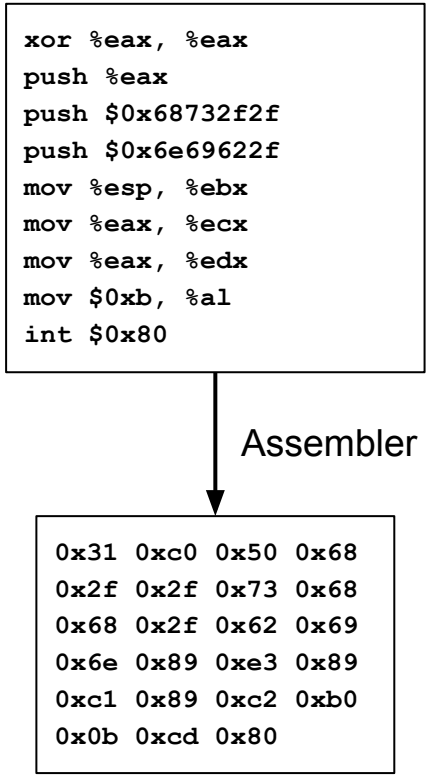
Note the NULL byte that terminates the string, automatically added by **gets**!




```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```

Writing Malicious Code

- The most common way of executing malicious code is to place it in memory yourself
  - Recall: Machine code is made of bytes
- **Shellcode**: Malicious code inserted by the attacker into memory, to be executed using a memory safety exploit
  - Called shellcode because it usually spawns a shell (terminal)
  - Could also delete files, run another program, etc.



Putting Together an Attack

1. Find a memory safety (e.g. buffer overflow) vulnerability
2. Write malicious shellcode at a known memory address
3. Overwrite the RIP with the address of the shellcode
    - Often, the shellcode can be written and the RIP can be overwritten in the same function call (e.g. `gets`), like in the previous example
4. Return from the function
5. Begin executing malicious shellcode

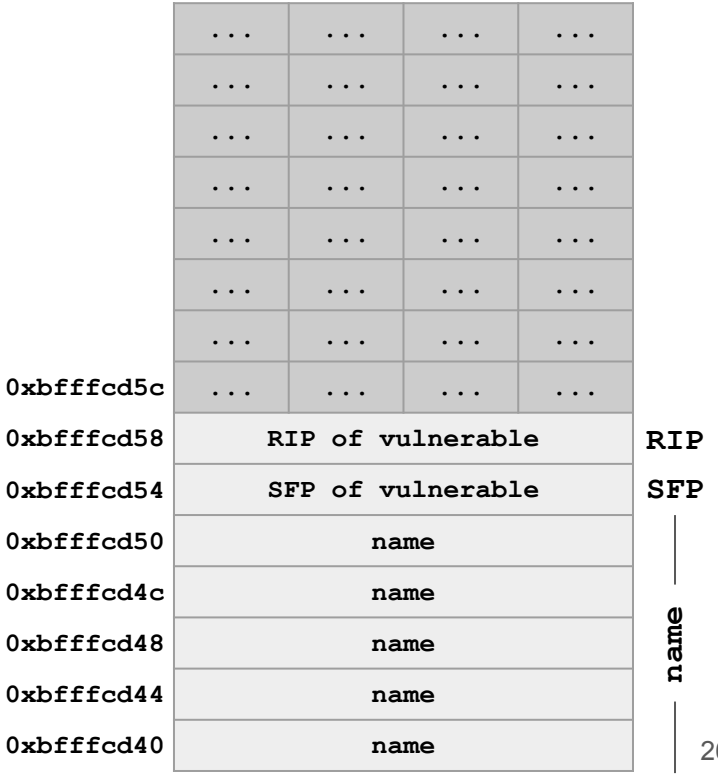
Constructing Exploits

Let **`SHELLCODE`** be a 12-byte shellcode. Assume that the address of **`name`** is **`0xbfffcd40`**.

What values should the attacker write in memory? Where should the values be written?

What should an attacker supply as input to the **`gets`** function?

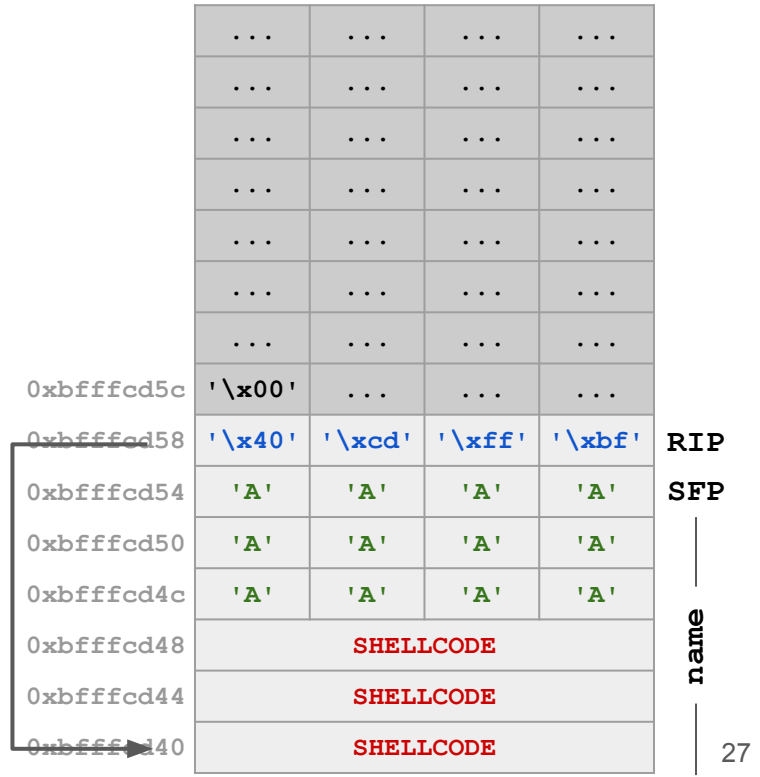
```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```



Constructing Exploits

- Input: `SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf' + '\n'`
  - 12 bytes of shellcode
  - 12 garbage bytes to overwrite the rest of **name** and the SFP of **vulnerable**
  - The address of where we placed the shellcode

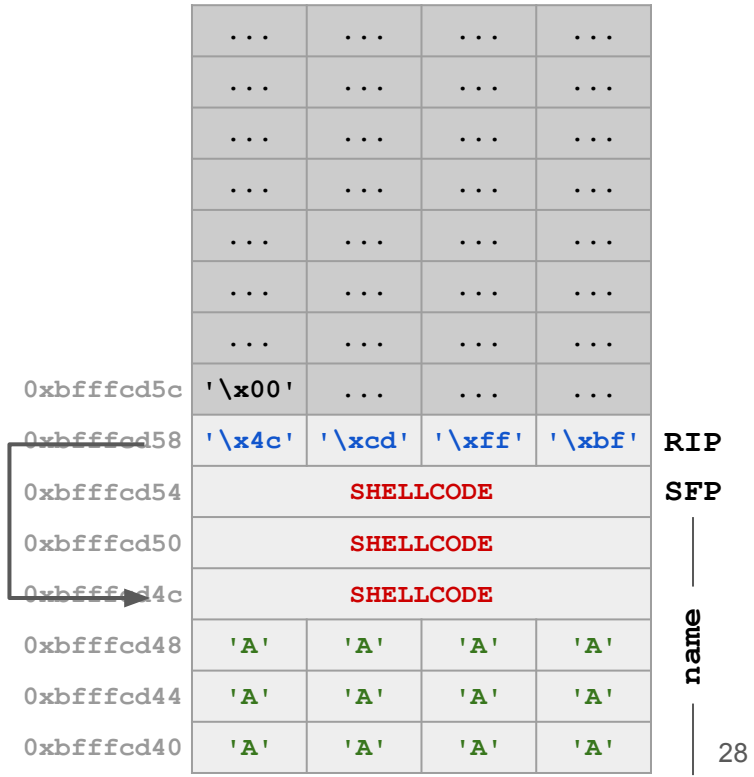
```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```



Constructing Exploits

- Alternative: `'A' * 12 + SHELLCODE + '\x4c\xcd\xff\xbf' + '\n'`
  - The address changed! Why?
    - We placed our shellcode at a different address (`name + 12`)!

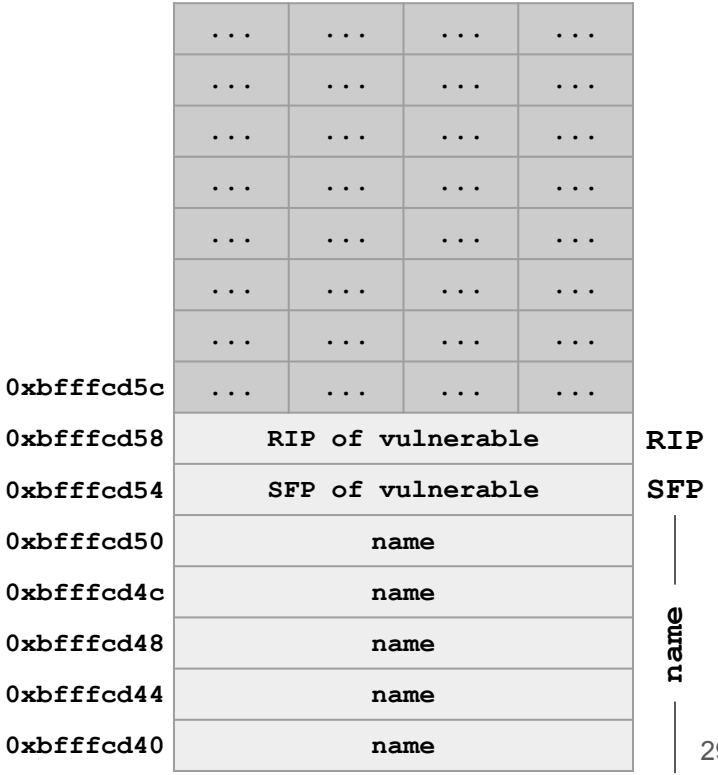
```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```



Constructing Exploits

What if the shellcode is too large? Now let **SHELLCODE** be a 28-byte shellcode. What should the attacker input?

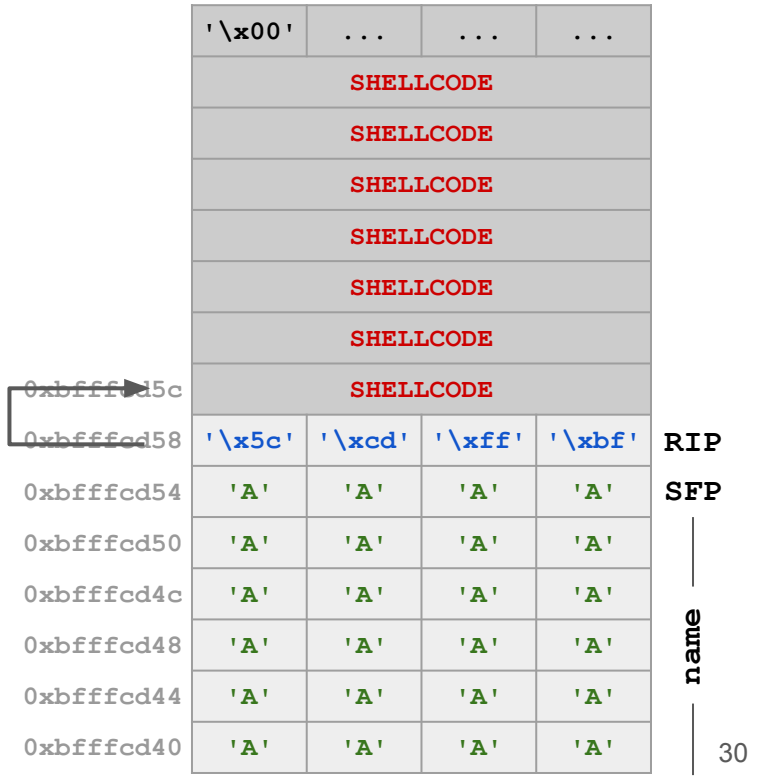
```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```



Constructing Exploits

- Solution: Place the shellcode *after* the RIP!
  - This works because **`gets`** lets us write as many bytes as we want
  - What should the address be?
- Input: **`'A' * 24 +`**
  `'\x5c\xcd\xff\xbf' + SHELLCODE`
  - 24 bytes of garbage
  - The address of where we placed the shellcode
  - 28 bytes of shellcode

```
void vulnerable(void) {
    char name[20];
    gets(name);
}
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

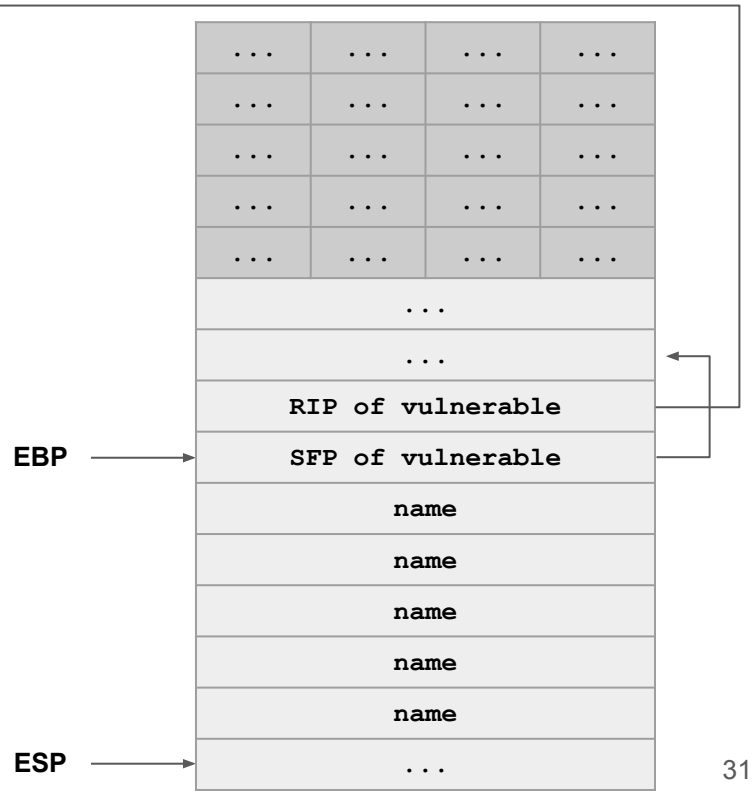
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

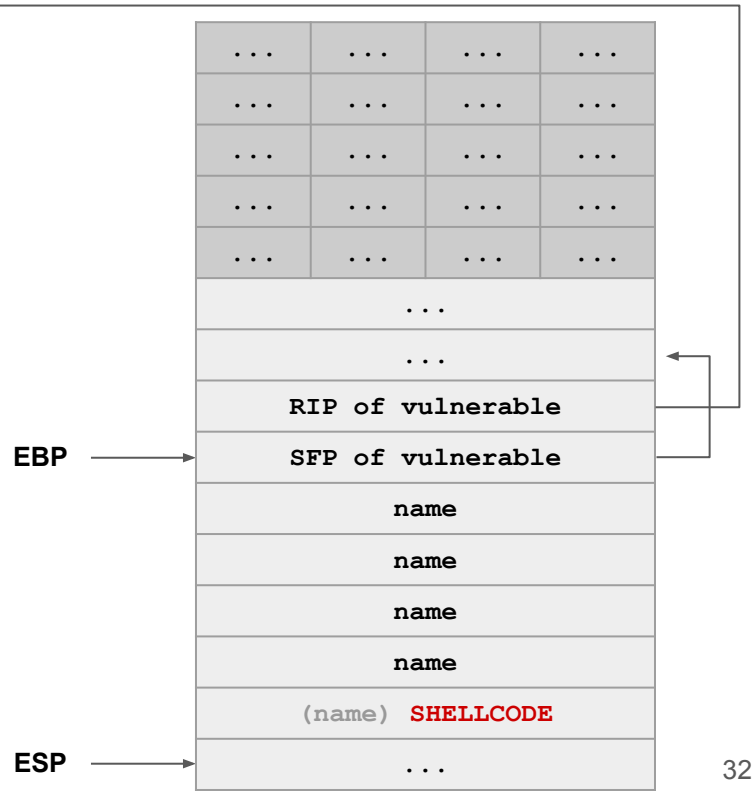
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

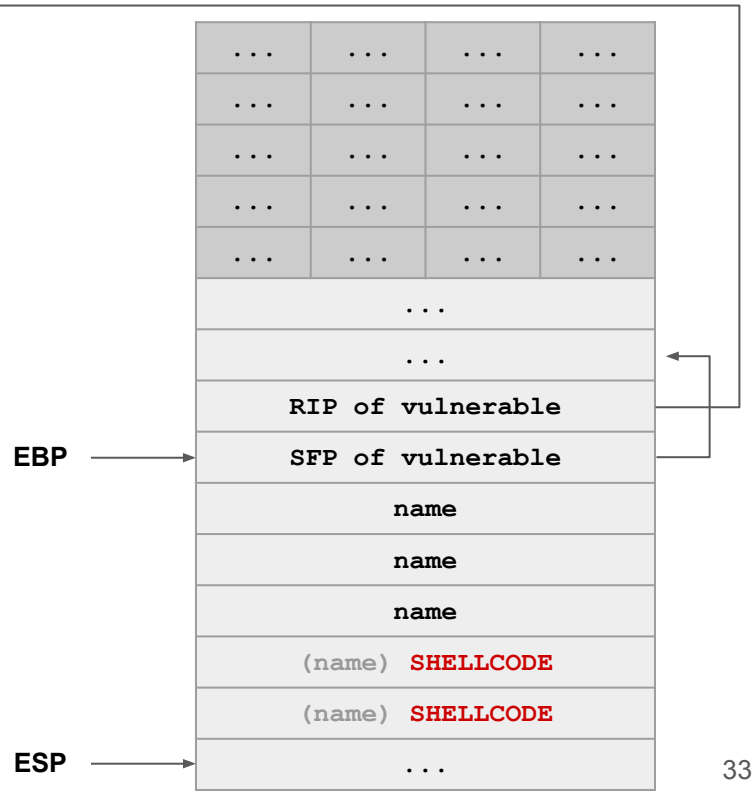
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

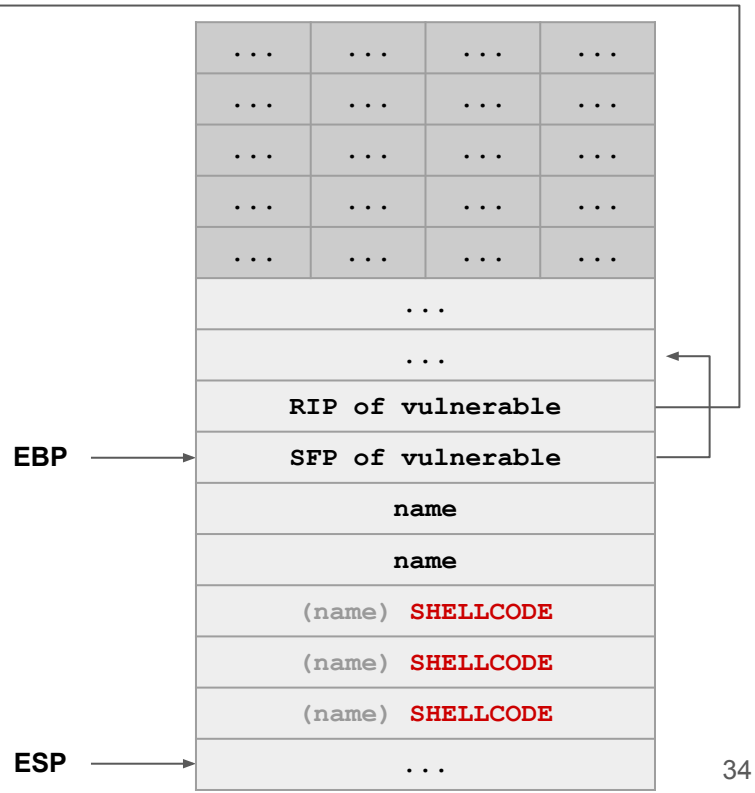
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

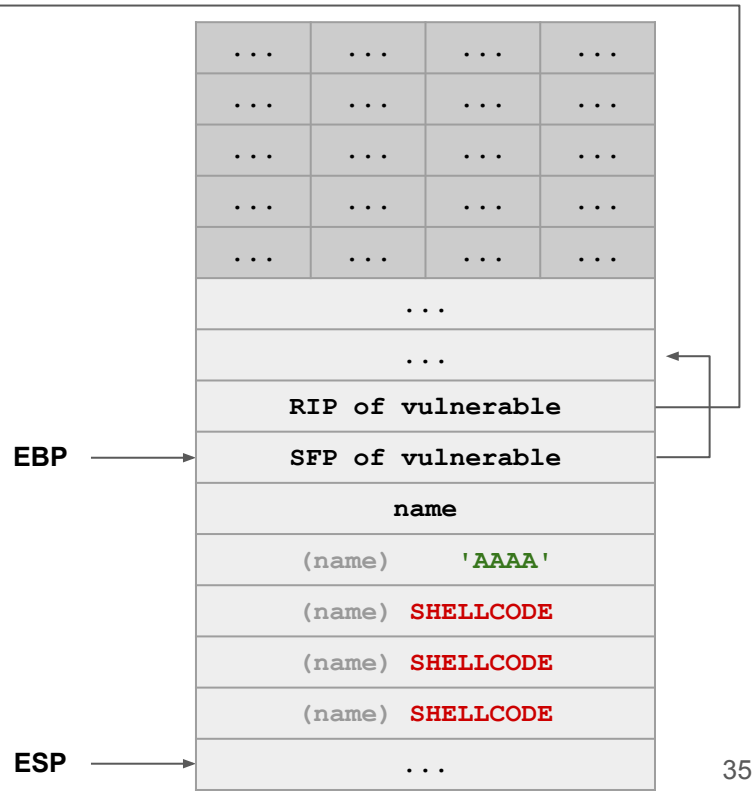
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow

Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

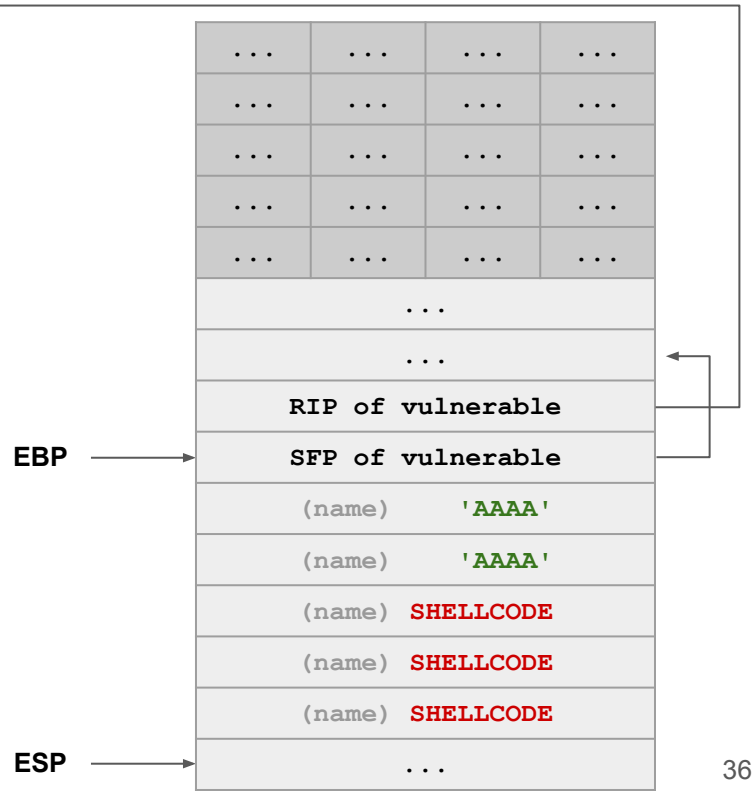
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



Walking Through a Buffer Overflow



Input:
`SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf'`

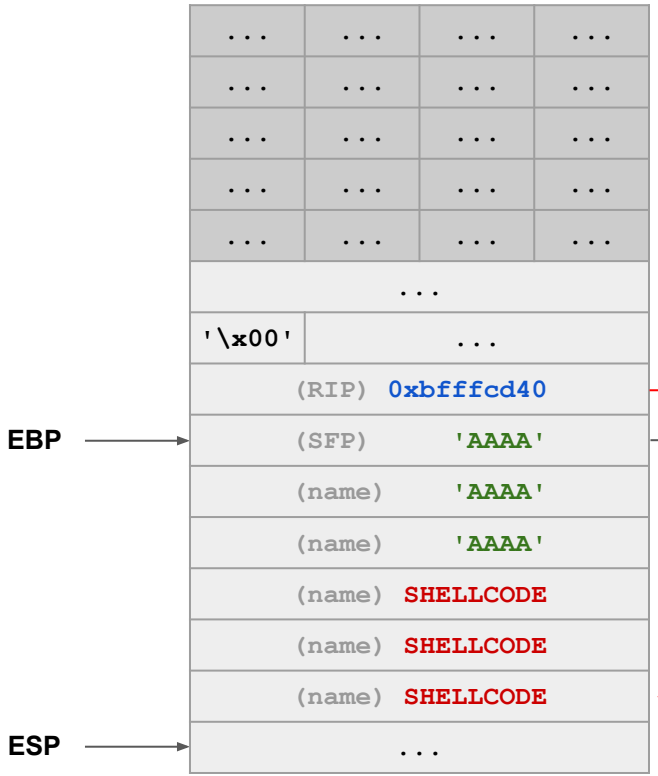
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



We overwrite the RIP (saved EIP) with the address of our shellcode `0xbfffcd40`, so the RIP is now pointing at our shellcode! Remember, this value will be restored to EIP (the instruction pointer) later.

Walking Through a Buffer Overflow



Input:
`SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf'`

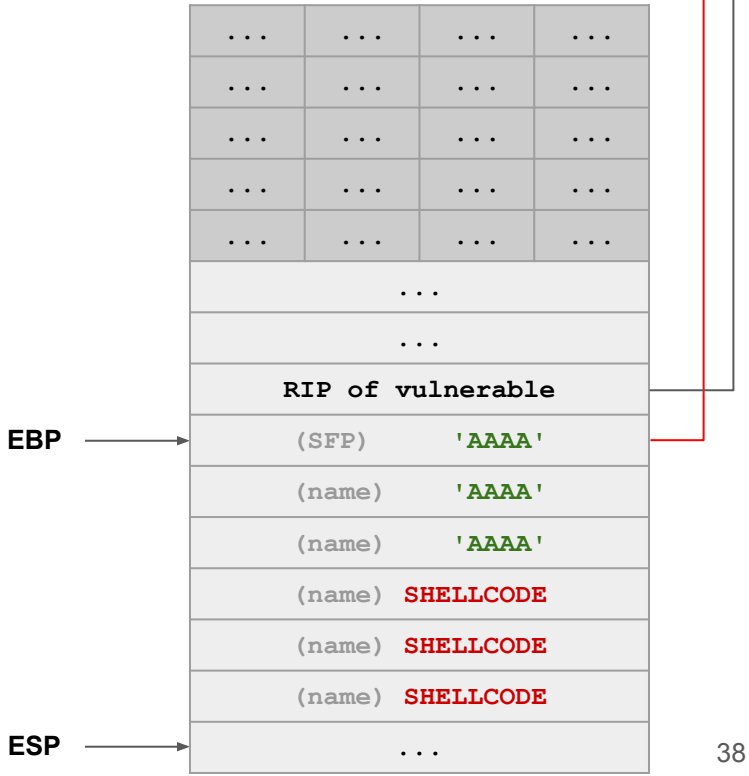
```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```



We overwrite the SFP (saved EBP) with `'AAAA'`, so the SFP is now pointing at the (probably invalid) address `AAAA` (`0x41414141`)

Walking Through a Buffer Overflow



Input:

```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

```
void vulnerable(void) {
    char name[20];
    gets(name);
}

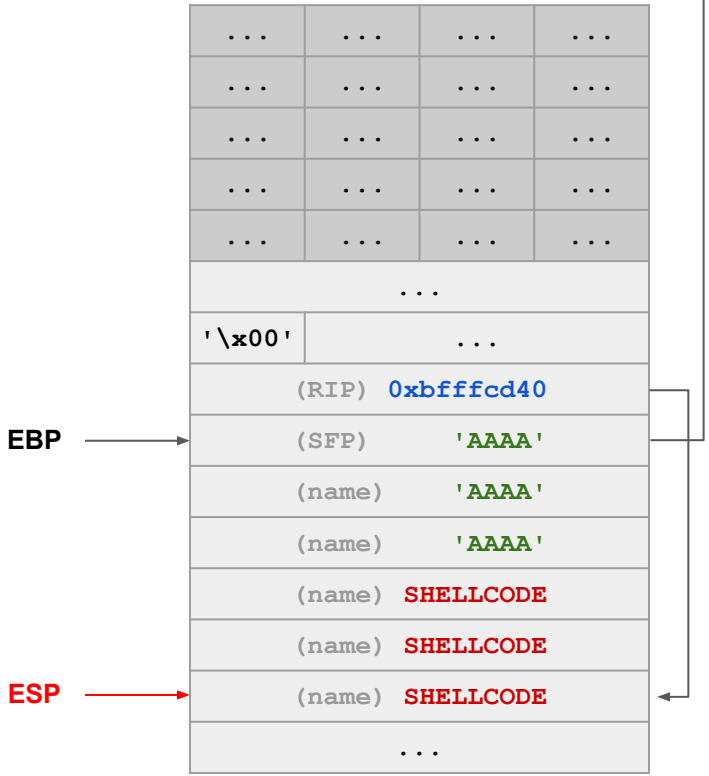
int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```

Returning from **gets**: Move ESP up by 4.



Walking Through a Buffer Overflow



Input:
`SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf'`

```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```

ESP → EBP →

...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...
...			
'\x00'	...		
(RIP)	0xbfffcd40		
(SFP)	'AAAA'		
(name)	'AAAA'		
(name)	'AAAA'		
(name)	SHELLCODE		
(name)	SHELLCODE		
(name)	SHELLCODE		
...			

Function epilogue: Move ESP to EBP.

Walking Through a Buffer Overflow



Input:
`SHELLCODE + 'A' * 12 + '\x40\xcd\xff\xbf'`

```
void vulnerable(void) {
    char name[20];
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

EIP →

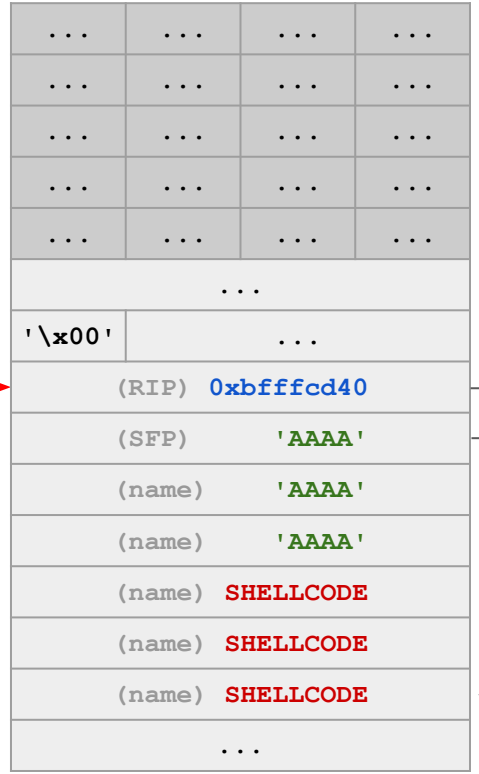
```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```

Function epilogue: Restore the SFP into EBP. We overwrote SFP to `'AAAA'`, so the EBP now also points to the address `'AAAA'`. We don't really care about EBP, though.

EBP →

ESP →



Walking Through a Buffer Overflow

Input:
```
SHELLCODE + 'A' * 12 +
'\x40\xcd\xff\xbf'
```

```
void vulnerable(void) {
    char name[20];
    gets(name);
}

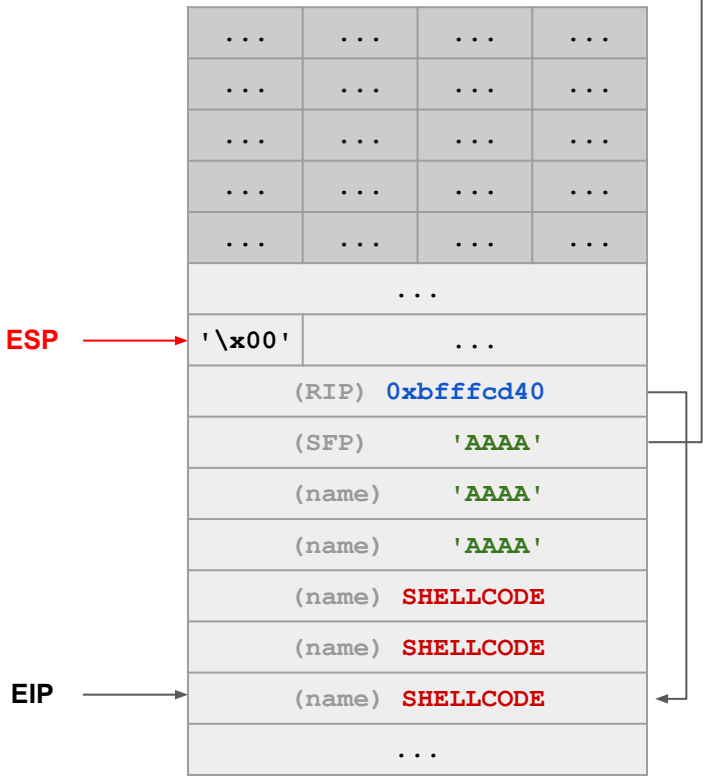
int main(void) {
    vulnerable();
    return 0;
}
```

```
vulnerable:
    ...
    call gets
    addl $4, %esp
    movl %ebp, %esp
    popl %ebp
    ret

main:
    ...
    call vulnerable
    ...
```

Function epilogue: Restore the RIP into EIP. We overwrote RIP to the address of shellcode, so the EIP (instruction pointer) now points to our shellcode!





Walking Through a Buffer Overflow



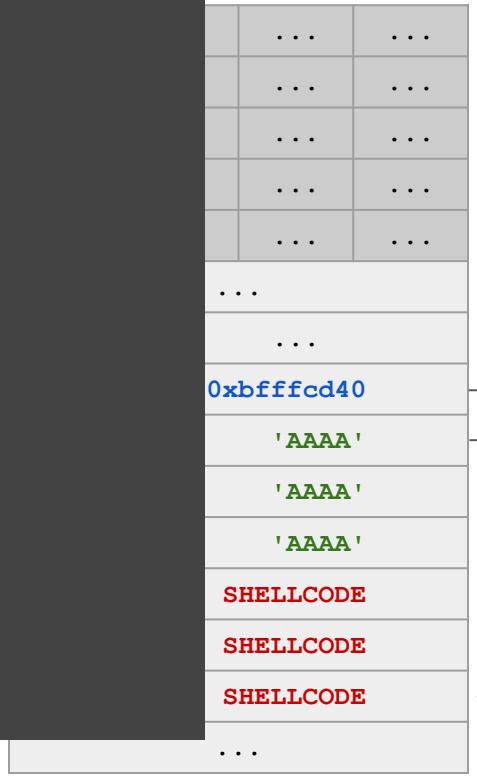
Inpu

```
SHELLCODE +
'\x40\xcd\
```

```
void vulnerable(v
    char name[20]
    gets(name);
}

int main(void) {
    vulnerable();
    return 0;
}
```

```
sh # _
```



Memory-Safe Code

Still Vulnerable Code?


```
void vulnerable?(void) {
    char *name = malloc(20);
    ...
    gets(name);
    ...
}
```

Heap overflows are also vulnerable!

Solution: Specify the Size


```
void safe(void) {
    char name[20];
    ...
    fgets(name, 20, stdin);
    ...
}
```

The length parameter specifies the size of the buffer and won't write any more bytes—no more buffer overflows!

Warning: Different functions take slightly different parameters

Solution: Specify the Size


```
void safer(void) {
    char name[20];
    ...
    fgets(name, sizeof(name), stdin);
    ...
}
```

`sizeof` returns the size of the variable (does **not** work for pointers)

Vulnerable C Library Functions

- **`gets`** - Read a string from stdin
  - Use **`fgets`** instead
- **`strcpy`** - Copy a string
  - Use **`strncpy`** (more compatible, less safe) or **`strlcpy`** (less compatible, more safe) instead
- **`strlen`** - Get the length of a string
  - Use **`strnlen`** instead (or **`memchr`** if you really need compatible code)
- … and more (look up C functions before you use them!)
  - **`man`** pages are your friend!

man fgets

ashmchiu — less ‹ man fgets — 169×61

```
FGETS(3)                                Library Functions Manual                                FGETS(3)

NAME
     fgets, gets — get a line from a stream

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <stdio.h>

     char *
     fgets(char * restrict str, int size, FILE * restrict stream);

     char *
     gets(char *str);

DESCRIPTION
     The fgets() function reads at most one less than the number of characters specified by size from the given stream and stores them in the string str.  Reading
     stops when a newline character is found, at end-of-file or error.  The newline, if any, is retained.  If any characters are read and there is no error, a '\0'
     character is appended to end the string.

     The gets() function is equivalent to fgets() with an infinite size and a stream of stdin, except that the newline character (if any) is not stored in the string.
     It is the caller's responsibility to ensure that the input line, if any, is sufficiently short to fit in the string.

RETURN VALUES
     Upon successful completion, fgets() and gets() return a pointer to the string.  If end-of-file occurs before any characters are read, they return NULL and the
     buffer contents remain unchanged.  If an error occurs, they return NULL and the buffer contents are indeterminate.  The fgets() and gets() functions do not
     distinguish between end-of-file and error, and callers must use feof(3) and ferror(3) to determine which occurred.

ERRORS
     [EBADF]            The given stream is not a readable stream.

     The function fgets() may also fail and set errno for any of the errors specified for the routines fflush(3), fstat(2), read(2), or malloc(3).

     The function gets() may also fail and set errno for any of the errors specified for the routine getchar(3).

SECURITY CONSIDERATIONS
     The gets() function cannot be used securely.  Because of its lack of bounds checking, and the inability for the calling program to reliably determine the length
```

Integer Memory Safety Vulnerabilities

Textbook Chapter 3.4

Signed/Unsigned Vulnerabilities

Is this safe?

```
void func(int len, char *data) {
    char buf[64];
    if (len > 64)
        return;
    memcpy(buf, data, len);
}
```

`int` is a **signed** type, but `size_t` is an **unsigned** type. What happens if `len == -1`?

This is a **signed** comparison, so `len > 64` will be false. But when we call memcpy(), casting **-1** to an unsigned type yields `0xffffffff`: another buffer overflow!

```
void *memcpy(void *dest, const void *src, size_t n);
```

Signed/Unsigned Vulnerabilities

Now this is an **unsigned** comparison, and no casting is necessary!

```
void safe(size_t len, char *data) {
    char buf[64];
    if (len > 64)
        return;
    memcpy(buf, data, len);
}
```

Integer Overflow Vulnerabilities

Is this safe?

What happens if `len == 0xffffffff`?

```
void func(size_t len, char *data) {
    char *buf = malloc(len + 2);
    if (buf == NULL)
        return;
    memcpy(buf, data, len);
    buf[len] = '\n';
    buf[len + 1] = '\0';
}
```

`len + 2 == 1`, enabling a heap overflow!

Integer Overflow Vulnerabilities


```
void safe(size_t len, char *data) {
    if (len > SIZE_MAX - 2)
        return;
    char *buf = malloc(len + 2);
    if (buf == NULL)
        return;
    memcpy(buf, data, len);
    buf[len] = '\n';
    buf[len + 1] = '\0';
}
```

It's clunky, but you need to check bounds whenever you add to integers!

Integer Overflows in the Wild

WJXT Jacksonville

Link

Broward Vote-Counting Blunder Changes Amendment Result

November 4, 2004

The Broward County Elections Department has egg on its face today after a computer glitch misreported a key amendment race, according to WPLG-TV in Miami.

Amendment 4, which would allow Miami-Dade and Broward counties to hold a future election to decide if slot machines should be allowed at racetracks, was thought to be tied. But now that a computer glitch for machines counting absentee ballots has been exposed, it turns out the amendment passed.

"The software is not geared to count more than 32,000 votes in a precinct. So what happens when it gets to 32,000 is the software starts counting backward," said Broward County Mayor Ilene Lieberman.

That means that Amendment 4 passed in Broward County by more than 240,000 votes rather than the 166,000-vote margin reported Wednesday night. That increase changes the overall statewide results in what had been a neck-and-neck race, one for which recounts had been going on today. But with news of Broward's error, it's clear amendment 4 passed.

Integer Overflows in the Wild

- 32,000 votes is very close to 32,768, or 2^{15} (the article probably rounded)
  - Recall: The maximum value of a signed, 16-bit integer is $2^{15} - 1$
  - This means that an integer overflow would cause -32,768 votes to be counted!
- **Takeaway**: Check the limits of data types used, and choose the right data type for the job
  - If writing software, consider the largest possible use case.
    - 32 bits might be enough for Broward County but isn't enough for everyone on Earth!
    - 64 bits, however, would be plenty.

Another Integer Overflow in the Wild



9 to 5 Linux

Link

New Linux Kernel Vulnerability Patched in All Supported Ubuntu Systems, Update Now

Marius Nestor *January 19, 2022*

Discovered by William Liu and Jamie Hill-Daniel, the new security flaw (CVE-2022-0185) is an integer underflow vulnerability found in Linux kernel's file system context functionality, which could allow an attacker to crash the system or run programs as an administrator.

How Does This Vulnerability Work?

- The entire kernel (operating system) patch:

```
- if (len > PAGE_SIZE - 2 - size)
+ if (size + len + 2 > PAGE_SIZE)
    return invalf(fc, "VFS: Legacy: Cumulative options too
large)
```

- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?

How Does This Vulnerability Work?

- The entire kernel (operating system) patch:

```
- if (len > PAGE_SIZE - 2 - size)
+ if (size + len + 2 > PAGE_SIZE)
      return invalf(fc, "VFS: Legacy: Cumulative options too
large)
```

- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?
  - `PAGE_SIZE`, `size`, and `len` are unsigned

Summary: Memory Safety Vulnerabilities

- **Buffer overflows**: An attacker overwrites unintended parts of memory
  - **Stack smashing**: An attacker overwrites saved registers on the stack
  - **Memory-safe code**: Fixing code to avoid buffer overflows
- **Integer memory safety vulnerabilities**: An attacker exploits how integers are represented in C memory

How Does This Vulnerability Work?

- The entire kernel (operating system) patch:

```
- if (len > PAGE_SIZE - 2 - size)
+ if (size + len + 2 > PAGE_SIZE)
      return invalf(fc, "VFS: Legacy: Cumulative options too
large)
```

- Why was the original code (`len > PAGE_SIZE - 2 - size`) vulnerable?
  - `PAGE_SIZE`, `size`, and `len` are unsigned
  - If `size` is larger than `PAGE_SIZE`, then `PAGE_SIZE - 2 - size` will trigger a negative overflow to `0xFFFFFFFF`
- Result: An attacker can bypass the length check and write data into the kernel